8-2 · DAG 이해 및 유즈케이스 연구 · 실습 1H

유즈케이스 연구와 태스크 분해 설계

실무 사례 다섯을 훑고 우리 파이프라인을 쪼갠다

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

1시간 뒤에는

① 실무 유즈케이스 다섯의 공통점을 한 문장으로 말한다

② 우리 파이프라인을 태스크 넷으로 나누고 멱등 여부를 적는다

③ pipeline_skeleton 골격 DAG 를 만들어 순서대로 도는지 본다

④ 태스크 분해안 문서를 채워 8-3 의 출발점으로 남긴다

여기서 만든 골격과 문서가 8-3 실습의 설계도이자 8주차 산출물의 일부입니다

실습 흐름

다섯 단계 — 사례에서 골격 DAG 와 문서까지

7주차에 손으로 하던 일을 태스크로 쪼갭니다 — 팀별로 다르게 나눠도 됩니다, 근거만 있으면

1. 유즈케이스	2. 태스크 분해	3. 골격 DAG	4. 실행 확인	5. 문서화
실무 사례 5 공통점 찾기	넷으로 쪼개고 멱등한가	print 만 모양 먼저	unpause·trigger 순서 확인	분해안 표 근거 한 줄

1단계

실무 유즈케이스 다섯 — Airflow 가 하는 일

유즈케이스	주기	Airflow 가 하는 일
배치 ETL(야간 정산)	매일 새벽	수집→정제→적재 순서 보장, 실패 단계만 재시도
ML 모델 재학습	매주	데이터 준비→학습→평가→배포를 조건부로
리포트 자동 발송	매일 아침	집계 완료를 기다렸다가 생성·발송
데이터 품질 점검	적재 후	건수·널·중복 검사, 미달이면 다음 단계 중단
RAG 지식베이스 갱신 (우리 과정)	매일	새 문서 수집→MinIO 보관→검증 →임베딩→VectorDB 갱신

공통점 — 전부 『여러 단계가 순서대로』이고 『중간이 실패할 수 있다』 — 오케스트레이션이 필요한 조건

2단계

우리 파이프라인을 네 태스크로 — 멱등한가

태스크	하는 일	멱등한가
collect	투입구의 새 문서를 MinIO raw-docs 에 올린다	같은 키에 덮어씀 → 예
validate	문서가 비었거나 너무 짧으면 실패	읽기만 → 예
build_index	build_kb.py 로 잘라 임베딩해 lecture_kb 갱신	문서마다 지우고 다시 넣기 → 예
report	지식베이스 조각 수를 남긴다	읽기만 → 예

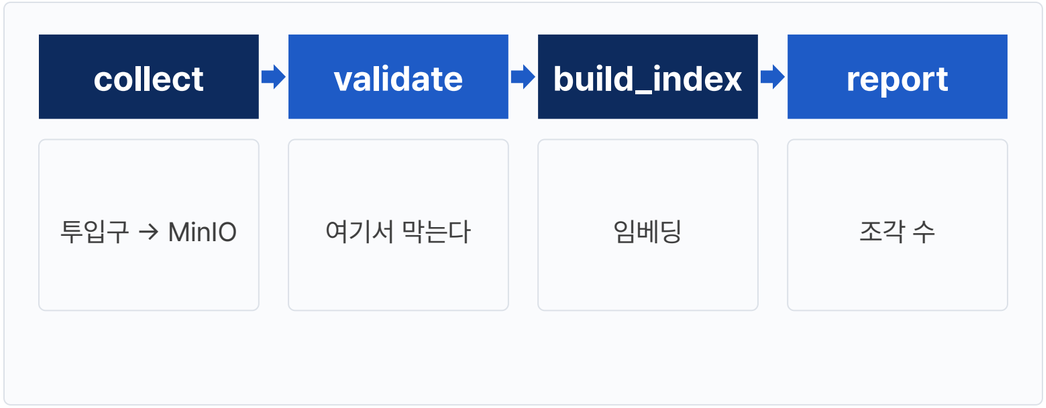
7-4 에서 MinIO 에 올리고 7-5 의 build_kb.py 로 lecture_kb 를 다시 만들던 일 — 8-3 에서 실제로 만들 분해안

2단계

의존성 초안과 나눈 근거

의존성 초안 — 한 줄

collect >> validate >> build_index >> report



VS

왜 이렇게 나눴나

예시일 뿐 — 팀마다 달라도 근거만 있으면 된다

- 임베딩이 가장 오래 걸린다 — 거기만 다시
- validate 를 따로 — 나쁜 문서를 막는다
- Spark 변환은 8-3 6단계에서 모양만

3단계

골격 DAG — print 만 넣고 모양부터 맞춘다

1. **STEPS = ["collect", "validate", "build_index", "report"]**
   8-3 에서 채울 네 이름

2. **make(name) 이 print 함수를 돌려준다**
   "[{name}] 아직 내용 없음" — 내용은 8-3 에서

3. **t = {s: PythonOperator(task_id=s, python_callable=make(s)) for s in STEPS}**
   딕셔너리 하나로 태스크 넷

4. **af dags show pipeline_skeleton | grep -- "→"**
   collect → validate · validate → build_index · build_index → report

> **마지막 줄 t["collect"] >> t["validate"] >> t["build_index"] >> t["report"] · 저장 뒤 af dags reserialize**

4단계

실행해 순서를 확인하세요

1. **af dags unpause pipeline_skeleton**
   새 DAG 는 기본이 일시정지

2. **af dags trigger pipeline_skeleton**
   sleep 40 뒤 상태를 본다

3. **af tasks states-for-dag-run pipeline_skeleton "$RUN" -o plain**
   네 줄 모두 success · start_date 가 collect → report 순서(실측)

골격 파일은

지우지 말 것

8-3 설계도

5단계

설계안을 문서로 — 8-3 의 출발점이자 8주차 산출물

1. **표 — 태스크 · 하는 일 · 멱등 근거 · 재시도**
   collect · validate · build_index · report 네 행
2. **의존성 ___ >> ___ >> ___ >> ___ · 스케줄 · catchup: False**
   골격 DAG 의 마지막 줄과 같아야 한다
3. **나눈 근거 · 실패했을 때 어디부터 다시 돌리나**
   태스크마다 한 줄씩
4. **문서를 지우면 지식베이스는 어떻게 되나 (예상)**
   이 표의 구멍 — 8-3 에서 실측으로 맞춰 본다

> **표를 채우고 근거를 적으면 오늘 몫이 끝 — 마지막 칸은 예상만**

확인 포인트

이 1시간이 끝날 때 확인할 것

1. 유즈케이스 다섯의 공통점을 한 문장으로 말했다
2. 태스크 넷마다 「멱등한가」와 근거가 적혀 있다
3. dags show 에 화살표 셋이 설계와 같이 나왔다
4. 네 태스크가 모두 success 이고 순서대로 돌았다
5. 분해안 문서를 채우고 「문서를 지우면」 예상을 적었다

8-2 · 실습 1H 마무리

다음은 정리·회고 (0.5H)

DAG 작성 규칙과 af 명령을 한 장으로 모으고, 자주 겪는 오류를 짚은 뒤 8-3 준비를 확인합니다

- ✓ **DAG 선언·순서·재시도·af 명령 요약표**
- ✓ **체크리스트 스스로 확인**
- ✓ **DAG 가 안 보일 때·회색일 때·catchup**
- ✓ **8-3 준비 — pipeline_skeleton.py**

가져갈 것

분해안 문서와 pipeline_skeleton.py